

IntelliSQL: Intelligent SQL Querying with LLMs

Using Gemini Pro

1. INTRODUCTION

1.1 Project Overview

IntelliSQL is an AI-powered system that converts natural language queries into SQL statements and retrieves results from a database. The system uses a Large Language Model to understand user questions written in plain English and translate them into structured SQL queries that can be executed on a database.

The application provides an easy way for users who do not have strong SQL knowledge to interact with databases. By integrating Natural Language Processing with database querying, IntelliSQL simplifies data exploration and retrieval.

The system is developed using Python, a lightweight SQLite database, and an AI model accessed through the Gemini API. The user interface is built using Streamlit to provide an interactive web application.

1.2 Purpose

The main purpose of IntelliSQL is to simplify database querying for users who are not familiar with SQL syntax. It enables users to ask questions in natural language and automatically generates SQL queries to fetch the required data.

The project aims to:

- Reduce the complexity of writing SQL queries
- Enable natural language interaction with databases
- Improve efficiency in retrieving data
- Demonstrate the application of Generative AI in database systems

2. IDEATION PHASE

2.1 Problem Statement

Many users struggle with writing SQL queries to retrieve information from databases. Non-technical users such as business analysts or students may not have sufficient knowledge of SQL syntax, which limits their ability to access data efficiently.

Therefore, there is a need for a system that allows users to interact with databases using natural language instead of writing complex SQL commands.

2.2 Empathy Map Canvas

Users interacting with databases often face difficulties in remembering SQL syntax and writing correct queries.

Users:

- Think: Writing SQL queries is complex and time-consuming.
- Feel: Frustrated when queries produce errors.
- Say: “I wish I could ask the database questions directly.”
- Do: Spend time searching for correct SQL syntax.

IntelliSQL addresses these issues by allowing users to ask questions in plain English.

2.3 Brainstorming

Several ideas were explored during the brainstorming phase to simplify database interaction. The idea of using Natural Language Processing and Generative AI models to translate English queries into SQL commands was identified as the most effective solution.

This approach combines artificial intelligence with database management systems to provide intelligent query assistance.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. User opens the IntelliSQL web application.
2. User enters a question in natural language.
3. The system sends the input to the AI model.
4. The AI model converts the question into an SQL query.
5. The SQL query is executed on the database.
6. Results are displayed to the user in table format.

3.2 Solution Requirement

Functional Requirements

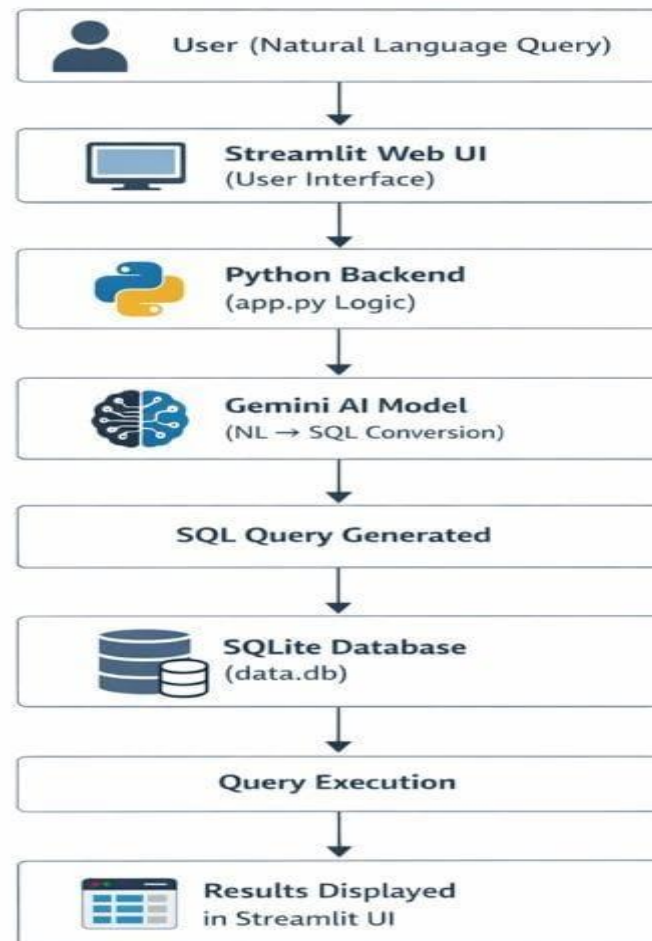
- Accept natural language queries from users
- Convert English queries to SQL queries
- Execute SQL queries on a database
- Display results in an easy-to-understand format

Non-Functional Requirements

- Fast query processing
- Secure API key management

- User-friendly interface
- Reliable database operations

3.3 Data Flow Diagram



3.4 Technology Stack

Programming Language: Python

Web Framework: Streamlit

Database: SQLite3

AI Model: Gemini API

Libraries

- streamlit
- google-generativeai
- sqlite3

4. PROJECT DESIGN

4.1 Problem Solution Fit

The project addresses the difficulty users face while writing SQL queries by enabling natural language interaction with databases. By leveraging AI models, IntelliSQL bridges the gap between human language and structured query language.

4.2 Proposed Solution

The proposed solution is a web-based system that allows users to input questions in natural language. The system uses a pre-trained AI model to convert the input into SQL queries, executes them on the database, and displays the results.

4.3 Solution Architecture

The system architecture consists of the following components:

User Interface:

Built using Streamlit to allow users to interact with the system.

AI Model Integration:

The Gemini API processes natural language queries and generates SQL statements.

Database Layer:

SQLite database stores and retrieves student information.

Application Logic:

Python scripts process input, generate SQL queries, execute database operations, and display results.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was completed in the following phases:

Phase 1 – Requirement Analysis

Understanding the problem and identifying system requirements.

Phase 2 – Database Creation

Creating the SQLite database and inserting sample records.

Phase 3 – AI Model Integration

Connecting the Gemini API and developing the natural language to SQL conversion logic.

Phase 4 – Web Application Development

Building the user interface using Streamlit.

Phase 5 – Testing and Deployment

Testing the system functionality and running the application.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The system was tested with different natural language queries to verify whether the AI model correctly generates SQL queries.

Example Tests

Input:

“How many students are there?”

Generated SQL:

```
SELECT COUNT(*) FROM Students;
```

Input:

“Show students working in INFOSYS”

Generated SQL:

```
SELECT * FROM Students WHERE Company='INFOSYS';
```

The system successfully generated correct SQL queries and displayed accurate results.

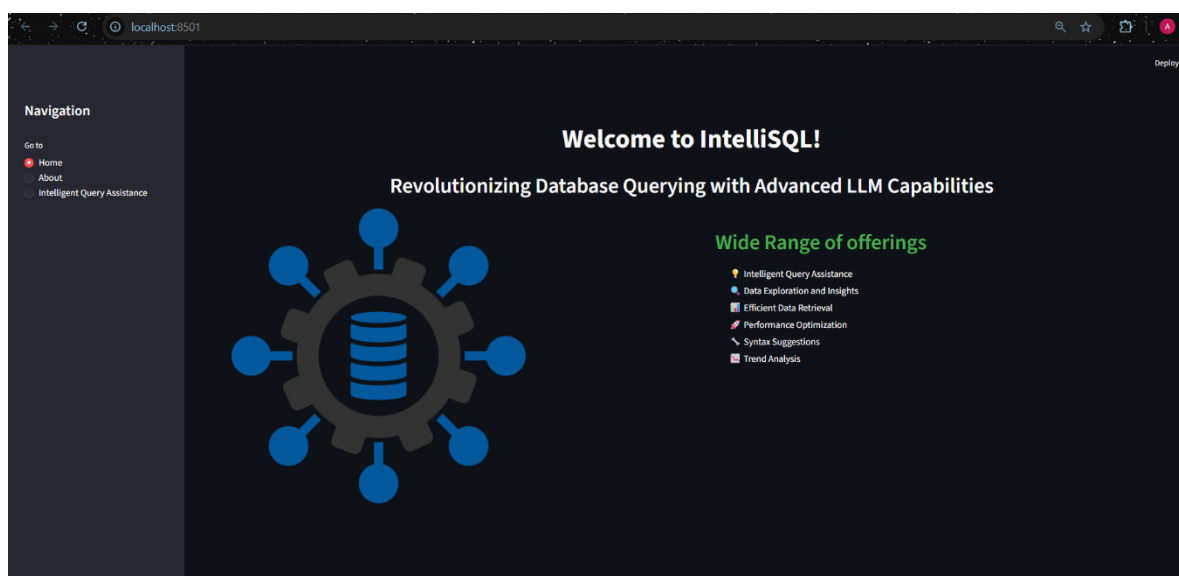
7. RESULTS

7.1 Output Screenshots

The application provides multiple pages:

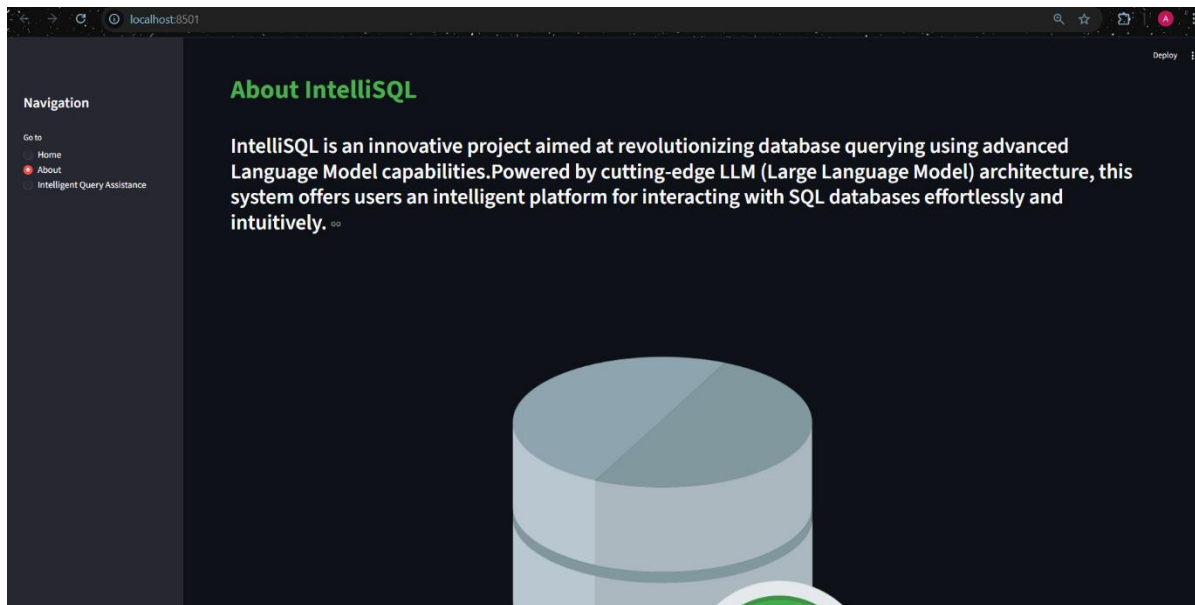
Home Page

Displays project introduction and system features.



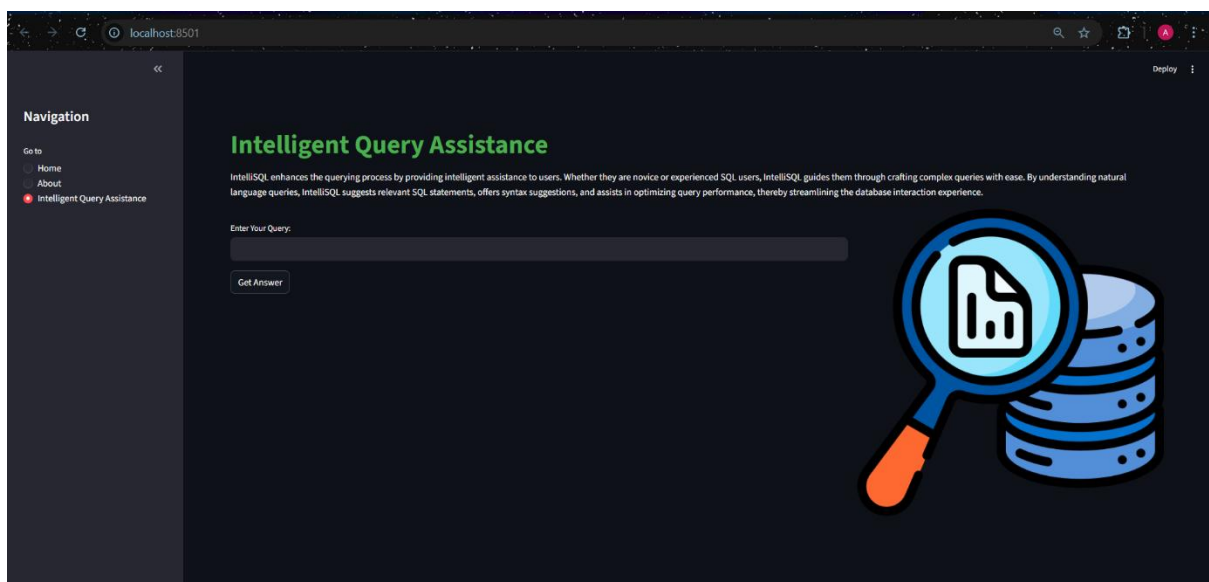
About Page

Provides details about the IntelliSQL project.



Query Assistance Page

Allows users to enter natural language queries and view generated SQL queries.



Database Results

Displays records retrieved from the database.

The screenshot shows a web application titled "Intelligent Query Assistance" running on localhost:8501. The interface includes a navigation sidebar with links to Home, About, and Intelligent Query Assistance. The main content area features a text input field for a query, a "Get Answer" button, and a display of the generated SQL query. The query result is shown as a table with 5 rows and 5 columns.

Intelligent Query Assistance

IntelliSQL enhances the querying process by providing intelligent assistance to users. Whether they are novice or experienced S

Enter Your Query:

show all the records

Get Answer

Generated SQL Query: `SELECT * FROM STUDENTS;`

The Response is:

	0	1	2	3
0	Sijo	BTech	75	JSW
1	Lijo	MTech	69	TCS
2	Rijo	BSc	79	WIPRO
3	Sibin	MSc	89	INFOSYS
4	Dilsha	MCom	99	Cyient

The screenshot shows the same web application with a different query entered: "how many students are working in the TCS". The generated SQL query is `SELECT COUNT(*) FROM STUDENTS WHERE Company="TCS";`. The response is a single row in a table with two columns: an index and a value.

Intelligent Query Assistance

IntelliSQL enhances the querying process by providing intelligent assistance to users. Whether they are novice or experienced S

Enter Your Query:

how many students are working in the TCS

Get Answer

Generated SQL Query: `SELECT COUNT(*) FROM STUDENTS WHERE Company="TCS";`

The Response is:

	value
0	1

8. ADVANTAGES & DISADVANTAGES

Advantages

- Simplifies database interaction
- No need for SQL knowledge

- Faster data retrieval
- User-friendly interface
- Useful for beginners and analysts

Disadvantages

- Accuracy depends on AI model responses
- Requires internet connection for API calls
- Limited to predefined database schema

9. CONCLUSION

IntelliSQL demonstrates how Artificial Intelligence can simplify database querying by converting natural language questions into SQL commands. The system successfully integrates Generative AI with database technologies to provide an intelligent querying solution.

The project highlights the potential of AI-powered tools in improving data accessibility and usability for both technical and non-technical users.

10. FUTURE SCOPE

Future improvements for IntelliSQL include:

- Support for multiple database systems
- Advanced query optimization
- Voice-based query input
- Improved natural language understanding
- Integration with large enterprise databases

11. APPENDIX

Source Code

requiremet.txt:

streamlit

google-genai

python-dotenv

sql.py:

```
import sqlite3
connection = sqlite3.connect("data.db")
cursor = connection.cursor()
table="CREATE TABLE Students(name VARCHAR(30), class VARCHAR(10), marks INT,
company VARCHAR(30))"

cursor.execute(table)

cursor.execute("insert into Students values('Sijo', 'BTech', 75, 'JSW')")
cursor.execute("insert into Students values('Lijo', 'MTech', 69, 'TCS')")
cursor.execute("insert into Students values('Rijo', 'BSc', 79, 'WIPRO')")
cursor.execute("insert into Students values('Sibin', 'MSc', 89, 'INFOSYS')")
cursor.execute("insert into Students values('Dilsha', 'MCom', 99, 'Cyient')")

print("the inserted records-")
df = cursor.execute("select * from students")
for row in df:
    print(row)

connection.commit()
connection.close()
```

app.py:

```
import streamlit as st
import os
import sqlite3
from dotenv import load_dotenv
from google import genai

# Load environment variables
load_dotenv()

# Configure Gemini API using .env file
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))
prompt = [
    """
```

You are an expert in converting English questions to SQL query!

The SQL database has the name STUDENTS and has the following columns - NAME, CLASS, Marks, Company \n\n For example, \n Example 1 - How many entries of records are present?, the SQL command will be something like this: SELECT COUNT(*) FROM

STUDENTS \n Example 2 - Tell me all the students studying in MCom class?, the SQL command will be something like this: SELECT * FROM STUDENTS WHERE CLASS="MCom";

```
""""  
]
```

```
def get_response(que, prompt):  
    response = client.models.generate_content(  
        model="gemini-2.5-flash",  
        contents=[prompt[0], que]  
    )  
    #return response.text.strip()  
    sql_query = response.text.strip()  
  
    # Remove markdown formatting if present  
    sql_query = sql_query.replace("`sql", "")  
    sql_query = sql_query.replace("`", "")  
    sql_query = sql_query.strip()  
  
    return sql_query
```

```
def read_query(sql, db):  
    conn = sqlite3.connect(db)  
    cursor = conn.cursor()  
    cursor.execute(sql)  
    rows = cursor.fetchall()  
    conn.commit()  
    conn.close()  
    return rows
```

```
def page_home():  
    st.markdown("""  
        <style>  
            body {  
                background-color: #2E2E2E;  
            }  
            .main-title {  
                text-align: center;  
                color: #4CAF50; /* Green color for headings */  
                font-size: 1.5em;  
            }  
        """)
```

```

.sub-title {
  text-align: center;
  color: #4CAF50; /* Green color for headings */
  font-size: 1.5em;
}
.offerings {
  padding: 20px;
  color: white; /* White color for body text */
}
.offerings h2{
  color: #4CAF50; /* Green color for headings */
}
.offerings ul{
  list-style-type: none;
  padding: 0;
}
offerings li {
  margin: 10px 0;
  font-size: 18px;
}
.custom-sidebar {
  background-color: #2E2E2E;
  color: white;
}
</style>
"""', unsafe_allow_html=True)
st.markdown("<h1 class='main-title'>Welcome to IntelliSQL!</h1>",
unsafe_allow_html=True)
st.markdown("<h2 class='sub-title'>Revolutionizing Database Querying with Advanced
LLM Capabilities</h2>", unsafe_allow_html=True)

col1, col2 = st.columns([1, 1])

with col1:
  st.image("https://cdn1.iconfinder.com/data/icons/business-dual-color-glyph-set-
3/128/Data_warehouse-1024.png", width=500)

with col2:
  st.markdown("""
<div class='offerings'>
  <h2>Wide Range of offerings</h2>
  <ul>
    <li>💡 Intelligent Query Assistance</li>
    <li>🔍 Data Exploration and Insights</li>
    <li>📊 Efficient Data Retrieval</li>
    <li>🚀 Performance Optimization</li>
  """

```

```

        <li>🔧 Syntax Suggestions</li>
        <li>📉 Trend Analysis</li>
    </ul>
</div>
""", unsafe_allow_html=True)

```

```

def page_about():
    st.markdown("""
        <style>
            .content {
                color: white; /* white color for body text */
            }
        </style>
        """, unsafe_allow_html=True)
    st.markdown("<h1 style='color: #4CAF50;'>About IntelliSQL</h1>",
        unsafe_allow_html=True)
    st.markdown("<div class='content'>", unsafe_allow_html=True)
    st.markdown("""
        <h2>IntelliSQL is an innovative project aimed at revolutionizing database querying using
        advanced Language Model capabilities. Powered by cutting-edge LLM (Large Language
        Model) architecture, this system offers users an intelligent platform for interacting with SQL
        databases effortlessly and intuitively.</h2>
        """, unsafe_allow_html=True)
    st.markdown("</div>", unsafe_allow_html=True)

```

```

    st.image("https://download.logo.wine/logo/Oracle_SQL_Developer/Oracle_SQL_Develop
er-Logo.wine.png", width="stretch")

```

```

def page_intelligent_query_assistance():
    st.markdown("""
        <style>
            .tool-input {
                margin-bottom: 20px;
                color: white; /* white color for body text */
            }
            .response {
                margin-top: 20px;
                color: white; /* white color for body text */
            }
        </style>
        """, unsafe_allow_html=True)

```

```

    st.markdown("<h1 style='color: #4CAF50;'>Intelligent Query Assistance</h1>",
        unsafe_allow_html=True)

```

```
st.write("""
```

IntelliSQL enhances the querying process by providing intelligent assistance to users. Whether they are novice or experienced SQL users, IntelliSQL guides them through crafting complex queries with ease. By understanding natural language queries, IntelliSQL suggests relevant SQL statements, offers syntax suggestions, and assists in optimizing query performance, thereby streamlining the database interaction experience.

```
""")
```

```
col1, col2 = st.columns([2, 1])
```

```
with col1:
```

```
    st.markdown("<div class='tool-input'>", unsafe_allow_html=True)
```

```
    que = st.text_input("Enter Your Query:", key="sql_query")
```

```
    submit = st.button("Get Answer", key="submit_button", help="Click to retrieve the SQL data")
```

```
    st.markdown("</div>", unsafe_allow_html=True)
```

```
    if submit or que:
```

```
        try:
```

```
            response = get_response(que, prompt)
```

```
            st.write(f"*Generated SQL Query:* {response}")
```

```
            response = read_query(response, "data.db")
```

```
            st.markdown("<div class='response'>", unsafe_allow_html=True)
```

```
            st.subheader("The Response is:")
```

```
            st.table(response)
```

```
            st.markdown("</div>", unsafe_allow_html=True)
```

```
        except Exception as e:
```

```
            st.subheader("Error:")
```

```
            st.error(f" error occurred: {e}")
```

```
with col2:
```

```
    st.image("https://cdn-icons-png.flaticon.com/512/9850/9850877.png", width="stretch")
```

```
def main():
```

```
    st.set_page_config(
```

```
        page_title="IntelliSQL",
```

```
        page_icon="",
```

```
        layout="wide"
```

```
)
```

```
    st.sidebar.title("Navigation")
```

```
    st.sidebar.markdown("<style>.sidebar.sidebar-content {background-color: #2E2E2E; color: white;}</style>", unsafe_allow_html=True)
```

```
    pages = {
```

```
        "Home": page_home,
```

```
        "About": page_about,
```

```
        "Intelligent Query Assistance": page_intelligent_query_assistance,
```

```
}  
selection = st.sidebar.radio("Go to", list(pages.keys()))  
page = pages[selection]  
page()  
  
if __name__ == "__main__":  
    main()
```

Dataset

SQLite database created using sql.py.

GitHub Repository

<https://github.com/PathanAfrin20/IntelliSQL>